LAB 7 – FUZZ TESTING

Password Policy Checker | Hypothesis and Pytest

Name: Renetta Isabel Nathan

SRN: PES1UG24AM221

1. Objective

The objective of this lab was to apply fuzz testing to a Python password policy checker using the Hypothesis property-based testing library. The tests generate a range of input values, including empty strings and None, to expose input-validation defects that may not be detected by ordinary example-based testing. The application was then modified to handle the discovered edge cases more safely.

2. Application Overview

The assigned application contains three functions. first_character_type(pw) classifies the first password character as a digit, letter, or symbol. split_credentials(text) separates a username and password using a colon. digit_ratio(pw) calculates the fraction of characters in a password that are digits.

3. Original Application Execution

The original application was executed in Google Colab before making changes. The supplied run used the inputs “Bleh”, “Ren:900”, and “6788”. The application classified the first character as a letter, split the credentials into Ren and 900, and returned a digit ratio of 1.0 for 6788.

```
==== Running first_character_type ====
Enter a password: Bleh
First Character Type: letter

==== Running split_credentials ====
Enter user:password (e.g. alice:Secret1): Ren:900
User and Password: ('Ren', '900')

==== Running digit_ratio ====
Enter a password to measure: 6788
Digit Ratio: 1.0
```

4. Hypothesis Fuzz Testing and Bugs Found

The starter test file was completed with three Hypothesis tests. Each test generated arbitrary text or None and called one of the application functions. The tests checked that the functions did not raise unexpected exceptions. Pytest collected three tests, and all three failed against the original implementation.

Function	Input Found	Original Failure
first_character_type	Empty string ('') and None	IndexError; TypeError
split_credentials	Empty string ('') and None	ValueError; AttributeError
digit_ratio	Empty string ('') and None	ZeroDivisionError; TypeError

Hypothesis minimized the failing examples to simple edge cases. The first function attempted to access pw[0] when the string was empty and also attempted to index None. The credentials function expected exactly one colon-separated pair and failed when the input was empty or None. The digit-ratio function divided by the length of the password, which was zero for an empty string, and attempted to iterate over None.

5. Corrective Changes

The application was hardened with input validation and safer handling of malformed values:

Function	Correction Made
first_character_type	Checks that the input is a non-empty string. Invalid or empty input returns 'invalid'.
split_credentials	Checks for a string containing a colon, then uses partition(':') so additional colons remain in the password. Invalid input returns ('', '').
digit_ratio	Checks for a non-empty string before division. Invalid or empty input returns 0.0.

6. Execution of the Corrected Application

After updating password_policy.py, the application was executed again with the inputs “Password123”, “alice:Secret1”, and “Pass123”. It classified the first character as a letter, returned the username-password pair ('alice', 'Secret1'), and calculated a digit ratio of 0.375 for Pass123.

```
••• ==== Running first_character_type ====
    Enter a password: Password123
    First Character Type: letter

    ==== Running split_credentials ====
    Enter user:password (e.g. alice:Secret1): alice:Secret1
    User and Password: ('alice', 'Secret1')

    ==== Running digit_ratio ====
    Enter a password to measure:    Pass123
    Digit Ratio: 0.375
```

7. Re-running the Hypothesis Tests

The same test suite was executed against the corrected application using python -m pytest test_password_policy.py -v. All three tests passed. The supplied Pytest output reports 3 passed in 0.56 seconds.

```
!python -m pytest test_password_policy.py -v

============================= test session starts ==============================
platform linux -- Python 3.13.18, pytest-8.4.2, pluggy-1.7.0 -- /usr/bin/python3
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: /content/L6_L7_39_password_policy/lab7
plugins: hypothesis-6.170.15, anyio-4.16.0, langsmith-0.14.17, typeguard-4.6.0, platformdirs-4.14.3
collected 3 items

test_password_policy.py::test_first_character_type_no_crash PASSED       [ 33%]
test_password_policy.py::test_split_credentials_no_crash PASSED          [ 66%]
test_password_policy.py::test_digit_ratio_no_crash PASSED                [100%]

============================== 3 passed in 0.56s ===============================
```

8. Result and Conclusion

Fuzz testing successfully identified input-validation defects in all three functions. The discovered cases included empty strings and None values, which caused indexing errors, credential-splitting errors, iteration errors, and division by zero. Defensive checks and safer string parsing were added to the application. After these changes, the application ran with the supplied sample inputs and all three Hypothesis no-crash tests passed. The lab demonstrated how property-based fuzz testing can expose edge cases and help improve the robustness of a Python program.

9. Deliverables

The code deliverables for this repository are password_policy.py (the corrected application) and test_password_policy.py (the completed Hypothesis test file). This report contains the supplied execution and testing evidence.